

UNIVERSIDAD TECNOLÓGICA NACIONAL (UTN)

Tecnicatura Universitaria en Programación a Distancia

MATERIA: PROGRAMACIÓN 1

Sistema de Gestión de Países

Estudiantes: Franco Trombotto | Sofía Odello

Comisión: 25

Grupo: 97

Fecha de Entrega: Junio de 2026

Año Académico: 2026

Índice General

1. Introducción	3
2. Marco Teórico	3
2.1 Variables y Tipos de Datos	3
2.2 Listas y Estructuras Secuenciales	3
2.3 Diccionarios como Registros	4
2.4 Funciones y Modularización	4
2.5 Estructuras de Control de Flujo	4
2.6 Persistencia de Datos mediante CSV	4
3. Decisiones Técnicas y Arquitectura	5
3.1 Organización Modular del Sistema	5
3.2 Diagrama de Flujo Operacional	5
3.3 Ejemplos de Ejecución y Evidencia	6
4. Dificultades y Conclusiones	7
4.1 Obstáculos Técnicos y Soluciones	7
4.2 Conclusiones y Reflexión Grupal	7
5. Medios de Acceso y Repositorio	8

1. Introducción

El presente documento detalla la documentación académica y técnica asociada al desarrollo del Trabajo Práctico Integrador para la materia **Programación 1** dentro de la Tecnicatura Universitaria en Programación a Distancia de la Universidad Tecnológica Nacional. El principal propósito de este sistema de software consiste en proveer un entorno de consola eficiente, modular y controlado para la manipulación, administración, filtrado y cálculo de estadísticas sobre registros de países en el ámbito mundial.

La aplicación resuelve de forma nativa necesidades operacionales clásicas de los sistemas de información, tales como el registro indexado (altas), bajas, modificaciones, ordenamiento por múltiples criterios (alfabético, demográfico y territorial) y la generación de métricas analíticas descriptivas. El desarrollo destaca por el uso de almacenamiento persistente local bajo formato CSV y un diseño desacoplado que garantiza la extensibilidad del sistema.

2. Marco Teórico

El diseño del software se fundamenta en los paradigmas lógicos y estructuras de datos provistas por el lenguaje de programación Python, respaldados por los lineamientos de la bibliografía oficial de la cátedra.

2.1 Variables y Tipos de Datos

Las variables constituyen celdas de memoria orientadas al resguardo temporal de información operativa. En Python, el tipado es dinámico pero fuertemente definido. Para este sistema se emplearon objetos de tipo string (`str`) para almacenar cadenas de texto como el nombre del país o su continente; tipos enteros (`int`) para variables discretas como la cantidad de habitantes; y tipos de punto flotante (`float`) para métricas continuas de precisión como la superficie territorial.

2.2 Listas y Estructuras Secuenciales

Una lista es una estructura de datos secuencial, ordenada y de carácter mutable. Permite almacenar colecciones de elementos indexados numéricamente a partir del valor cero. Dentro de la aplicación, las listas actúan como el contenedor principal en memoria viva para gestionar el conjunto de países cargados desde el almacenamiento físico, facilitando su recorrido lógico a través de iteradores.

2.3 Diccionarios como Registros

Los diccionarios representan colecciones no estructuradas de pares llave-valor (`key-value`). A diferencia de las secuencias numéricas, la indexación se efectúa mediante claves de texto inmutables. El sistema utiliza diccionarios para modelar de forma semántica e intuitiva cada "registro" o registro de país de la siguiente manera: `{ 'nombre': str, 'poblacion': int, 'superficie': float, 'continente': str }`, lo que garantiza el acceso directo a los campos sin depender de posiciones físicas rígidas.

2.4 Funciones y Modularización

Una función es un bloque de código reutilizable y delimitado diseñado para realizar una tarea computacional específica, favoreciendo la legibilidad y la reducción de redundancia. La modularización extiende este concepto al segmentar físicamente el programa en diversos archivos o módulos con responsabilidades lógicas aisladas e independientes, interactuando mediante directivas de importación (`import`).

2.5 Estructuras de Control de Flujo

El control del flujo de ejecución determina el orden en que se procesan las instrucciones bajo dos mecanismos primordiales:

- **Estructuras Condicionales (`if-elif-else`):** Permiten bifurcar los caminos de ejecución evaluando proposiciones lógicas y booleanas de verdadero o falso.
- **Estructuras Repetitivas o Bucles:** El ciclo `for` se utiliza para iteraciones definidas sobre elementos finitos (como las listas de países), mientras que el bucle `while` mantiene la ejecución indefinida de menús interactivos hasta que ocurra un evento de salida.

2.6 Persistencia de Datos mediante CSV

La persistencia define la propiedad de los datos de sobrevivir a la finalización del proceso de ejecución del software. El formato CSV (*Comma-Separated Values*) es un estándar tabular abierto altamente eficiente para representar datos tabulares. Su manipulación en este proyecto se realiza de acuerdo con las librerías nativas estándares, permitiendo resguardar y recuperar la base informativa de países de forma transparente.

3. Decisiones Técnicas y Arquitectura

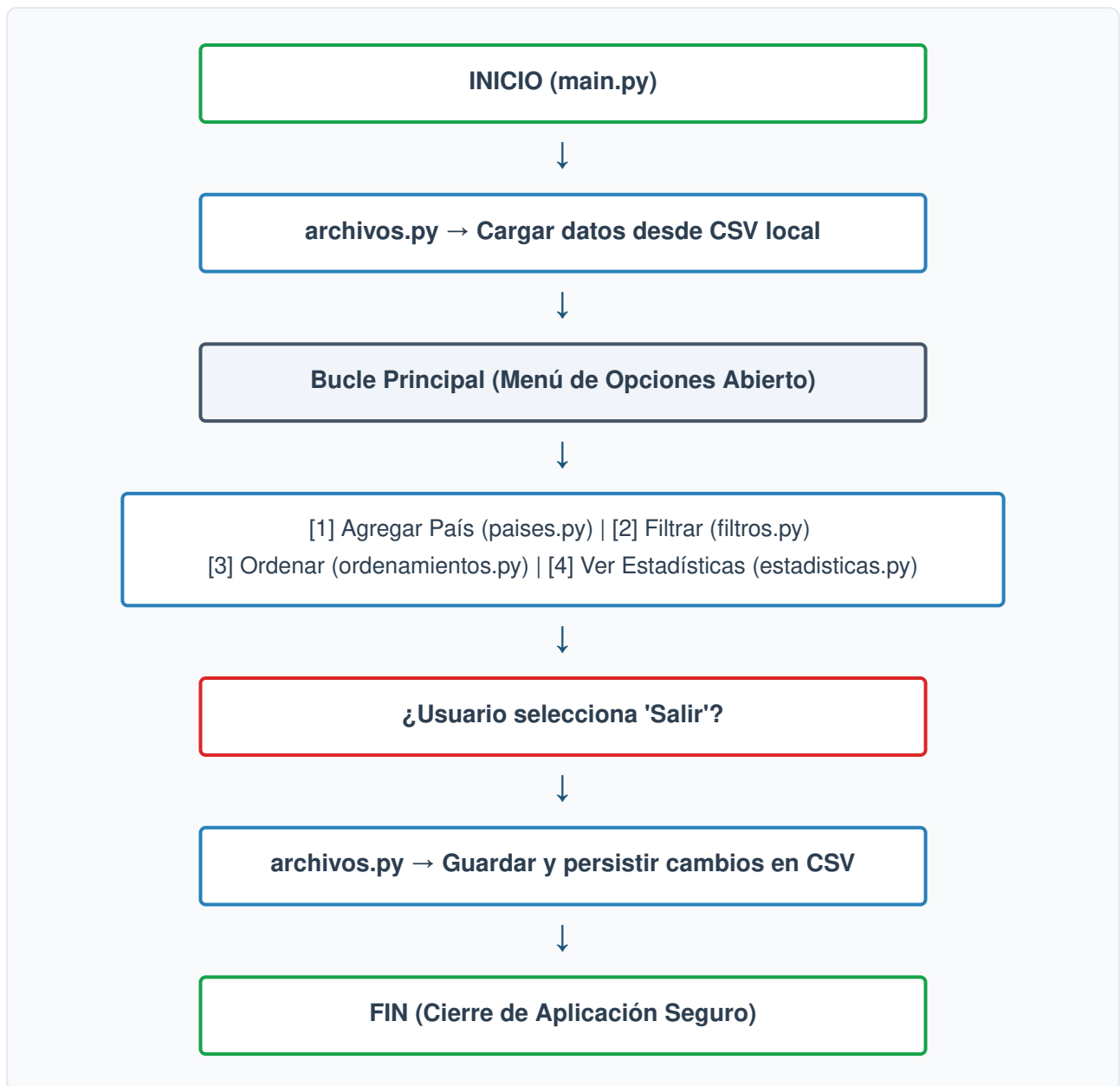
3.1 Organización Modular del Sistema

Con el objeto de aplicar el principio de separación de intereses y responsabilidad única, la solución arquitectónica se dividió en módulos desacoplados encargados de tareas atómicas:

- `archivos.py` : Administra de manera exclusiva la lectura y escritura en los discos de almacenamiento físico utilizando flujos CSV.
- `países.py` : Implementa el ABM básico (Altas, Bajas y Modificaciones) de las entidades de datos en memoria.
- `filtros.py` : Abstrae las funciones de emparejamiento lógico para aislar subconjuntos de información por continente o rangos.
- `ordenamientos.py` : Concentra las rutinas de ordenamiento de registros según los campos disponibles de la entidad.
- `estadisticas.py` : Procesa la lista de datos para derivar valores acumuladores (totales, promedios, máximos y mínimos).
- `submenus.py` : Construye la interfaz interactiva de consola y coordina las interacciones globales del flujo.
- `main.py` : Punto de entrada general de la aplicación.

3.2 Diagrama de Flujo Operacional

El siguiente esquema conceptual representa el ciclo de procesamiento y ejecución principal desde el inicio de la aplicación interactiva de consola:



Esquema 1.1: Flujo lógico secuencial y ramificaciones del sistema de información.

3.3 Ejemplos de Ejecución y Evidencia

A efectos de validar de forma práctica el correcto funcionamiento de las operaciones algorítmicas y la persistencia de los registros del sistema, se disponen las siguientes referencias visuales estructurales de ejecución:

```
===== MENU PRINCIPAL =====
1 - Mostrar países
2 - Agregar país
3 - Buscar país
4 - Filtros
5 - Ordenamientos
6 - Estadísticas
7 - Actualizar país
0 - Salir
```

Figura 1.1: Consola inicial del sistema desplegando las opciones de navegación unificadas para el usuario.

```
Elegí una opción: 4

--- FILTROS ---
1 - Por continente
2 - Por rango de población
3 - Por rango de superficie
0 - Volver
```

Figura 1.2: Salida tabular del procesamiento de filtros de datos geográficos y ordenación algorítmica.

```
Elegí una opción: 6

--- SUBMENÚ: ESTADÍSTICAS ---
1 - País con mayor población
2 - País con menor población
3 - Promedio de población
4 - Promedio de superficie
5 - Cantidad de países por continente
0 - Volver al Menú Principal
```

Figura 1.3: Bloque analítico desplegando los resultados estadísticos globales derivados de los registros cargados.

4. Dificultades y Conclusiones

4.1 Obstáculos Técnicos y Soluciones

El proceso de desarrollo iterativo expuso desafíos específicos que requirieron el análisis crítico de la arquitectura. En la siguiente tabla se resumen los problemas detectados y las acciones correctivas aplicadas para mitigarlos:

Dificultad Técnica	Acción Correctiva y Resolución
Curva de aprendizaje inicial en Git y control de versiones cooperativo	Se pautó el uso riguroso de ramas de trabajo (<i>feature branches</i>), revisiones de código cruzadas y la ejecución estructurada de comandos atómicos para la integración limpia de commits en GitHub.
Tipos de datos tipificados implícitamente como strings en el parseo CSV	Dado que el módulo de lectura CSV recupera de forma nativa cadenas textuales, se implementó una rutina de conversión explícita (<i>casting</i>) a variables numéricas (<code>int</code> y <code>float</code>) previas a la lógica aritmética.
Inestabilidad y caídas por ingreso de datos incorrectos (inputs inválidos)	Se blindó el ingreso de datos en la terminal mediante bloques de manejo de excepciones y errores <code>try-except ValueError</code> , asegurando que ante letras en campos numéricos el sistema notifique y repita la solicitud en lugar de colapsar.
Bloqueos lógicos por Importaciones Circulares de módulos	Al modularizar las llamadas, se generaron dependencias recursivas. Se resolvió mediante la refactorización de imports, trasladando llamadas específicas al interior de las funciones requeridas y aislando las responsabilidades en <code>main.py</code> .

4.2 Conclusiones y Reflexión Grupal

La consecución de este Trabajo Práctico Integrador permitió constatar de forma práctica las bondades del paradigma de programación modular. El diseño estructurado dividiendo el software en módulos específicos no solo simplifica las tareas de codificación, sino que emula las prácticas de desarrollo de software reales de la industria.

Como reflexión colectiva, el grupo destaca que las listas de diccionarios constituyen una alternativa nativa robusta y de excelente rendimiento para representar bases de datos relacionales simuladas en memoria viva. Asimismo, se revalorizó la trascendencia de las herramientas estándar como el formato CSV para proveer soluciones de persistencia eficientes y portables en proyectos medianos sin agregar sobrecargas de infraestructura tecnológica.

5. Medios de Acceso y Repositorio

A fines de garantizar la transparencia del proceso de desarrollo de software, la trazabilidad del código fuente mediante control de versiones y la validación funcional interactiva, se proveen los accesos oficiales al proyecto integrado:

Repositorio de Código Fuente (GitHub):

<https://github.com/francotrombotto/TPI-Programacion1>

Video Explicativo de Defensa del Proyecto:

https://youtu.be/gvRFR_gCbm8

El repositorio contiene los commits cronológicos ejecutados colaborativamente, las ramas de desarrollo y los archivos CSV utilizados como bases de datos de prueba para las ejecuciones del sistema.

6. Bibliografía

Se utilizó la bibliografía brindada por la cátedra.